

Module 1

Design Patterns and Principles

Name: Harikuntla Madhuri

Exercise 1: Implementing the Singleton Pattern

Scenario:

Ensure that a logging utility class has only one instance throughout the application lifecycle.

Code:

```
Main.java
1 class Logger {
2     private static Logger obj = null;
3
4     private Logger() { }
5
6     public static Logger getInstance() {
7         if (obj == null) {
8             obj = new Logger();
9         }
10        return obj;
11    }
12
13    public void log(String msg) {
14        System.out.println("LOG: " + msg);
15    }
16 }
17
18 class Main {
19     public static void main(String[] args) {
20         Logger l1 = Logger.getInstance();
21         Logger l2 = Logger.getInstance();
22
23         l1.log("app started");
24         l2.log("user logged in");
25
26         // should print true - same object
27         System.out.println("Same instance? " + (l1 == l2));
28     }
29 }
```

Output:

```
LOG: app started
LOG: user logged in
Same instance? true

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

Create different types of documents (Word, PDF, Excel) using the Factory Method Pattern.

Code:

```
Main.java
1 interface Document {
2     void open();
3 }
4
5 class WordDocument implements Document {
6     public void open() { System.out.println("Word doc opened"); }
7 }
8
9 class PdfDocument implements Document {
10     public void open() { System.out.println("PDF doc opened"); }
11 }
12
13 class ExcelDocument implements Document {
14     public void open() { System.out.println("Excel doc opened"); }
15 }
16
17 abstract class DocumentFactory {
18     public abstract Document createDocument();
19 }
20
21 class WordFactory extends DocumentFactory {
22     public Document createDocument() { return new WordDocument(); }
23 }
24
25 class PdfFactory extends DocumentFactory {
26     public Document createDocument() { return new PdfDocument(); }
27 }
28
29 class ExcelFactory extends DocumentFactory {
30     public Document createDocument() { return new ExcelDocument(); }
31 }
```

```

32
33 class Main {
34     public static void main(String[] args) {
35         DocumentFactory f = new WordFactory();
36         f.createDocument().open();
37
38         f = new PdfFactory();
39         f.createDocument().open();
40
41         f = new ExcelFactory();
42         f.createDocument().open();
43     }
44 }

```

Output:

```

Word doc opened
PDF doc opened
Excel doc opened

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 3: Implementing the Builder Pattern

Scenario:

Create complex objects like a Computer with multiple optional parts using the Builder Pattern.

Code:

```

Main.java
1 class Computer {
2     private String cpu;
3     private String ram;
4     private String hdd;
5     private String gpu;
6
7     private Computer(Builder b) {
8         cpu = b.cpu;
9         ram = b.ram;
10        hdd = b.hdd;
11        gpu = b.gpu;
12    }
13
14    public String toString() {
15        return "CPU=" + cpu + ", RAM=" + ram + ", HDD=" + hdd + ", GPU=" + gpu;
16    }
17
18    static class Builder {
19        private String cpu, ram, hdd, gpu;
20
21        public Builder setCpu(String c) { cpu = c; return this; }
22        public Builder setRam(String r) { ram = r; return this; }
23        public Builder setHdd(String h) { hdd = h; return this; }
24        public Builder setGpu(String g) { gpu = g; return this; }
25
26        public Computer build() { return new Computer(this); }
27    }
28 }
29

```

```

30 - class Main {
31 -     public static void main(String[] args) {
32 -         Computer c1 = new Computer.Builder()
33 -             .setCpu("i9 13th gen")
34 -             .setRam("32GB")
35 -             .setHdd("1TB SSD")
36 -             .setGpu("RTX 4080")
37 -             .build();
38 -
39 -         Computer c2 = new Computer.Builder()
40 -             .setCpu("i5 12th gen")
41 -             .setRam("8GB")
42 -             .setHdd("512GB")
43 -             .build();
44 -
45 -         System.out.println(c1);
46 -         System.out.println(c2);
47 -     }
48 - }

```

Output:

```

CPU=i9 13th gen, RAM=32GB, HDD=1TB SSD, GPU=RTX 4080
CPU=i5 12th gen, RAM=8GB, HDD=512GB, GPU=null

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 4: Implementing the Adapter Pattern

Scenario:

Integrate multiple third-party payment gateways with different interfaces using the Adapter Pattern.

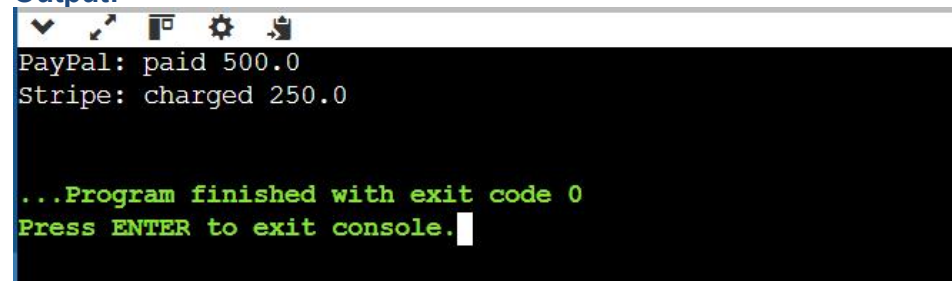
Code:

```

1 interface PaymentProcessor {
2     void processPayment(double amt);
3 }
4
5 class PayPalGateway {
6     public void makePayment(double amount) {
7         System.out.println("PayPal: paid " + amount);
8     }
9 }
10
11 class StripeGateway {
12     public void chargeCard(double amount) {
13         System.out.println("Stripe: charged " + amount);
14     }
15 }
16
17 class PayPalAdapter implements PaymentProcessor {
18     PayPalGateway pg;
19     public PayPalAdapter(PayPalGateway pg) { this.pg = pg; }
20     public void processPayment(double amt) { pg.makePayment(amt); }
21 }
22
23 class StripeAdapter implements PaymentProcessor {
24     StripeGateway sg;
25     public StripeAdapter(StripeGateway sg) { this.sg = sg; }
26     public void processPayment(double amt) { sg.chargeCard(amt); }
27 }
28
29 class Main {
30     public static void main(String[] args) {
31         PaymentProcessor p1 = new PayPalAdapter(new PayPalGateway());
32         p1.processPayment(500.0);
33
34         PaymentProcessor p2 = new StripeAdapter(new StripeGateway());
35         p2.processPayment(250.0);
36     }
37 }

```

Output:



```

PayPal: paid 500.0
Stripe: charged 250.0

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 5: Implementing the Decorator Pattern

Scenario:

Send notifications via multiple channels (Email, SMS, Slack) using the Decorator Pattern.

Code:


```

1 interface Notifier {
2     void send(String message);
3 }
4
5 class EmailNotifier implements Notifier {
6     public void send(String message) {
7         System.out.println("Email sent: " + message);
8     }
9 }
10
11 abstract class NotifierDecorator implements Notifier {
12     protected Notifier notifier;
13     public NotifierDecorator(Notifier n) { this.notifier = n; }
14     public void send(String message) { notifier.send(message); }
15 }
16
17 class SMSNotifierDecorator extends NotifierDecorator {
18     public SMSNotifierDecorator(Notifier n) { super(n); }
19     public void send(String message) {
20         super.send(message);
21         System.out.println("SMS sent: " + message);
22     }
23 }
24
25 class SlackNotifierDecorator extends NotifierDecorator {
26     public SlackNotifierDecorator(Notifier n) { super(n); }
27     public void send(String message) {
28         super.send(message);
29         System.out.println("Slack sent: " + message);
30     }
31 }
32
33 class Main {
34     public static void main(String[] args) {
35         Notifier n = new SlackNotifierDecorator(
36             new SMSNotifierDecorator(
37                 new EmailNotifier());
38         n.send("Server down!");
39     }
40 }

```

Output:

```

Email sent: Server down!
SMS sent: Server down!
Slack sent: Server down!

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 6: Implementing the Proxy Pattern

Scenario:

Load images from a remote server using the Proxy Pattern with lazy initialization and caching.

Code:

```

1 interface Image {
2     void display();
3 }
4
5 class RealImage implements Image {
6     private String fname;
7
8     public RealImage(String fname) {
9         this.fname = fname;
10        loadImage();
11    }
12
13    private void loadImage() {
14        System.out.println("Loading from server: " + fname);
15    }
16
17    public void display() {
18        System.out.println("Displaying: " + fname);
19    }
20 }
21
22 class ProxyImage implements Image {
23     private String fname;
24     private RealImage realImg = null;
25
26     public ProxyImage(String fname) { this.fname = fname; }
27
28     public void display() {
29         if (realImg == null) {
30             realImg = new RealImage(fname); // Lazy Load + cached
31         }
32         realImg.display();
33     }
34 }
35
36 class Main {
37     public static void main(String[] args) {
38         Image img = new ProxyImage("photo.jpg");
39
40         System.out.println("-- first call --");
41         img.display();
42
43         System.out.println("-- second call --");
44         img.display(); // no reload
45     }
46 }

```

Output:

```

-- first call --
Loading from server: photo.jpg
Displaying: photo.jpg
-- second call --
Displaying: photo.jpg

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 7: Implementing the Observer Pattern

Scenario:

Notify multiple clients whenever stock prices change using the Observer Pattern.

Code:

```
1- import java.util.ArrayList;
2- import java.util.List;
3-
4- interface Observer {
5-     void update(String stock, double price);
6- }
7-
8- interface Stock {
9-     void register(Observer o);
10-    void deregister(Observer o);
11-    void notifyObservers();
12- }
13-
14- class StockMarket implements Stock {
15-     private List<Observer> list = new ArrayList<>();
16-     private String stockName;
17-     private double price;
18-
19-     public void setPrice(String name, double p) {
20-         stockName = name;
21-         price = p;
22-         notifyObservers();
23-     }
24-
25-     public void register(Observer o) { list.add(o); }
26-     public void deregister(Observer o) { list.remove(o); }
27-
28-     public void notifyObservers() {
29-         for (Observer o : list) o.update(stockName, price);
30-     }
31- }
32-
33- class MobileApp implements Observer {
34-     public void update(String stock, double price) {
35-         System.out.println("[Mobile App] " + stock + " -> Rs." + price);
36-     }
37- }
38-
39- class WebApp implements Observer {
40-     public void update(String stock, double price) {
41-         System.out.println("[Web App] " + stock + " -> Rs." + price);
42-     }
43- }
44-
45- class Main {
46-     public static void main(String[] args) {
47-         StockMarket mkt = new StockMarket();
48-         mkt.register(new MobileApp());
49-         mkt.register(new WebApp());
50-
51-         mkt.setPrice("TCS", 3450.50);
52-         mkt.setPrice("INFY", 1725.00);
53-     }
54- }
```

Output:

```
✓ ↗ 📄 ⚙️ 🗑️
[Mobile App] TCS -> Rs.3450.5
[Web App] TCS -> Rs.3450.5
[Mobile App] INFY -> Rs.1725.0
[Web App] INFY -> Rs.1725.0

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 8: Implementing the Strategy Pattern

Scenario:

Select different payment methods (Credit Card, PayPal) at runtime using the Strategy Pattern.

Code:

```
1 interface PaymentStrategy {
2     void pay(double amount);
3 }
4
5 class CreditCardPayment implements PaymentStrategy {
6     public void pay(double amount) {
7         System.out.println("Paid Rs." + amount + " by credit card");
8     }
9 }
10
11 class PayPalPayment implements PaymentStrategy {
12     public void pay(double amount) {
13         System.out.println("Paid Rs." + amount + " via PayPal");
14     }
15 }
16
17 class PaymentContext {
18     private PaymentStrategy strategy;
19
20     public PaymentContext(PaymentStrategy s) { strategy = s; }
21
22     public void setStrategy(PaymentStrategy s) { strategy = s; }
23
24     public void doPayment(double amount) { strategy.pay(amount); }
25 }
26
27 class Main {
28     public static void main(String[] args) {
29         PaymentContext ctx = new PaymentContext(new CreditCardPayment());
30         ctx.doPayment(1500.0);
31
32         ctx.setStrategy(new PayPalPayment());
33         ctx.doPayment(800.0);
34     }
35 }
```

Output:

```
Paid Rs.1500.0 by credit card
Paid Rs.800.0 via PayPal

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 9: Implementing the Command Pattern

Scenario:

Issue commands to turn home automation devices on or off using the Command Pattern.

Code:

```
1 interface Command {
2     void execute();
3 }
4
5 class Light {
6     public void turnOn() { System.out.println("Light is ON"); }
7     public void turnOff() { System.out.println("Light is OFF"); }
8 }
9
10 class LightOnCommand implements Command {
11     Light light;
12     public LightOnCommand(Light l) { light = l; }
13     public void execute() { light.turnOn(); }
14 }
15
16 class LightOffCommand implements Command {
17     Light light;
18     public LightOffCommand(Light l) { light = l; }
19     public void execute() { light.turnOff(); }
20 }
21
22 class RemoteControl {
23     private Command cmd;
24     public void setCommand(Command c) { cmd = c; }
25     public void pressButton() { cmd.execute(); }
26 }
27
28 class Main {
29     public static void main(String[] args) {
30         Light light = new Light();
31         RemoteControl remote = new RemoteControl();
32
33         remote.setCommand(new LightOnCommand(light));
34         remote.pressButton();
35
36         remote.setCommand(new LightOffCommand(light));
37         remote.pressButton();
38     }
39 }
```

Output:

```
Light is ON
Light is OFF

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 10: Implementing the MVC Pattern

Scenario:

Manage student records using the MVC pattern with Model, View, and Controller classes.

Code:

```

1 class Student {
2     private String name;
3     private int id;
4     private String grade;
5
6     public Student(String name, int id, String grade) {
7         this.name = name; this.id = id; this.grade = grade;
8     }
9
10    public String getName() { return name; }
11    public int getId() { return id; }
12    public String getGrade() { return grade; }
13
14    public void setName(String n) { name = n; }
15    public void setGrade(String g) { grade = g; }
16 }
17
18 class StudentView {
19     public void displayStudentDetails(String name, int id, String grade) {
20         System.out.println("Name: " + name);
21         System.out.println("ID: " + id);
22         System.out.println("Grade: " + grade);
23         System.out.println("-----");
24     }
25 }
26

```

```

27 class StudentController {
28     private Student model;
29     private StudentView view;
30
31     public StudentController(Student s, StudentView v) {
32         model = s; view = v;
33     }
34
35     public void setStudentName(String n) { model.setName(n); }
36     public void setStudentGrade(String g) { model.setGrade(g); }
37
38     public void updateView() {
39         view.displayStudentDetails(
40             model.getName(), model.getId(), model.getGrade());
41     }
42 }
43
44 class Main {
45     public static void main(String[] args) {
46         Student s = new Student("Ravi", 21, "B+");
47         StudentView v = new StudentView();
48         StudentController ctrl = new StudentController(s, v);
49         ctrl.updateView();
50
51         ctrl.setStudentGrade("A");
52         ctrl.updateView();
53     }
54 }

```

Output:

```

Name: Ravi
ID: 21
Grade: B+
-----
Name: Ravi
ID: 21
Grade: A
-----

```

Exercise 11: Implementing Dependency Injection

Scenario:

Manage dependencies between a service class and repository class using constructor-based Dependency Injection.

Code:

```
1 interface CustomerRepository {
2     String findCustomerById(int id);
3 }
4
5 class CustomerRepositoryImpl implements CustomerRepository {
6     public String findCustomerById(int id) {
7         return "Customer #" + id + " -> Suresh Kumar";
8     }
9 }
10 class CustomerService {
11     private CustomerRepository repo;
12
13     // dependency injected via constructor
14     public CustomerService(CustomerRepository r) {
15         repo = r;
16     }
17
18     public String getCustomerInfo(int id) {
19         return repo.findCustomerById(id);
20     }
21 }
22 class Main {
23     public static void main(String[] args) {
24         CustomerRepository repo = new CustomerRepositoryImpl();
25         CustomerService svc = new CustomerService(repo);
26
27         System.out.println(svc.getCustomerInfo(101));
28         System.out.println(svc.getCustomerInfo(205));
29     }
30 }
```

Output:

```
Customer #101 -> Suresh Kumar
Customer #205 -> Suresh Kumar

...Program finished with exit code 0
Press ENTER to exit console.□
```